1. Amazon API Gateway

Question: How do you create a REST API using Amazon API Gateway that returns student information?
Procedure:

1. Log in to AWS Management Console.
2. Search for **API Gateway**.
3. Click **Create API**.
4. Select **REST API → Build**.
5. Enter API name, e.g. StudentAPI.
6. Create a resource such as /students.
7. Select /students → **Create Method → GET**.
8. Select the integration type, such as Lambda.
9. Select the required Lambda function.
10. Click **Deploy API**.
11. Create a stage such as dev or prod.
12. Copy the **Invoke URL**.
13. Test the API using Postman or browser.
    Flow: Client → API Gateway → Lambda/Backend → Database → Response

2. VPC

Question: How do you create a VPC for a three-tier web application?
Procedure:

1. Open **VPC Console**.
2. Click **Create VPC**.
3. Select **VPC only**.
4. Enter name, e.g. MyVPC.
5. Specify CIDR, e.g. 10.0.0.0/16.
6. Click **Create VPC**.
7. Create public, application and database subnets.
8. Create an **Internet Gateway**.
9. Attach the IGW to the VPC.
10. Create route tables.
11. Associate subnets with route tables.

12. Create security groups.
13. Launch EC2/RDS resources into appropriate subnets.
    Architecture: Internet → IGW → Public Subnet → Private App Subnet → Private DB Subnet

3. Subnet

Question: How do you create public and private subnets?
Procedure:

1. Open **VPC → Subnets**.
2. Click **Create subnet**.
3. Select the VPC.
4. Enter subnet name.
5. Select an Availability Zone.
6. Enter CIDR, e.g. 10.0.1.0/24.
7. Click **Create subnet**.
8. Repeat for private subnet, e.g. 10.0.2.0/24.
9. Associate the public subnet with a public route table.
10. Associate the private subnet with a private route table.
11. Enable auto-assign public IPv4 only where required.

4. Routing Table

Question: How do you configure a routing table to make a subnet public?
Procedure:

1. Open **VPC → Route Tables**.
2. Click **Create route table**.
3. Enter name, e.g. PublicRouteTable.
4. Select the VPC.
5. Open **Routes → Edit routes**.
6. Add 0.0.0.0/0 → Internet Gateway.
7. Save the route.
8. Open **Subnet associations**.
9. Associate the public subnet.
   Private route: 0.0.0.0/0 → NAT Gateway

5. NAT Gateway

Question: How do you provide Internet access to EC2 instances in a private subnet?
Procedure:

1. Create a public subnet.
2. Create and attach an Internet Gateway.
3. Create a public route table.
4. Add 0.0.0.0/0 → Internet Gateway.
5. Associate the public subnet.
6. Go to **NAT Gateways**.
7. Click **Create NAT Gateway**.
8. Select the public subnet.
9. Allocate an **Elastic IP**.
10. Create the NAT Gateway.
11. Open the private route table.
12. Add 0.0.0.0/0 → NAT Gateway.
13. Associate the private subnet with this route table.
    Flow: Private EC2 → NAT Gateway → Internet Gateway → Internet

6. AWS KMS

Question: How do you create a KMS key and use it for encryption?
Procedure:

1. Open **AWS KMS**.
2. Select **Customer managed keys**.
3. Click **Create key**.
4. Select **Symmetric** key.
5. Add an alias, e.g. MyDatabaseKey.
6. Configure key administrators.
7. Configure users/roles allowed to use the key.
8. Review the key policy.
9. Create the key.
10. Use the key with services such as RDS, S3, EBS or DynamoDB.
11. Ensure IAM and KMS permissions are correctly configured.
    Remember: IAM Permissions + KMS Key Policy = Access Control

7. Amazon RDS

Question: How do you create an Amazon RDS MySQL database?
Procedure:

1. Open **Amazon RDS**.
2. Click **Create database**.
3. Select **Standard create**.
4. Select **MySQL**.
5. Select the required engine version.
6. Enter DB identifier, username and password.
7. Select instance class and storage.
8. Select the VPC.
9. Select the DB subnet group.
10. Configure the security group.
11. Set database port, normally 3306.
12. Enable KMS encryption if required.
13. Click **Create database**.
14. Wait until status becomes **Available**.
15. Copy the RDS endpoint.
16. Connect using MySQL/client/application.
    Flow: Application/EC2 → Security Group → RDS

8. DynamoDB

Question: How do you create a DynamoDB table and insert records?
Procedure:

1. Open **DynamoDB**.
2. Click **Create table**.
3. Enter table name, e.g. Students.
4. Define partition key, e.g. StudentID.
5. Select key type, e.g. String.
6. Select capacity mode: On-demand or Provisioned.
7. Click **Create table**.
8. Open the table.
9. Select **Explore table items**.
10. Click **Create item**.
11. Add attributes such as StudentID, Name, Course and Marks.
12. Save the item.

13. Use AWS SDK/API for CRUD operations.

9. CloudFormation

Question: How do you create AWS infrastructure automatically using CloudFormation?
Procedure:

1. Open **AWS CloudFormation**.
2. Click **Create stack**.
3. Select **Template is ready**.
4. Upload or specify the template.
5. Enter a stack name.
6. Configure parameters if required.
7. Review the resources.
8. Acknowledge IAM capabilities if required.
9. Click **Create stack**.
10. Monitor the **Events** tab.
11. Check the **Resources** tab after completion.
    Remember: Template → CloudFormation Stack → AWS Resources

10. CloudTrail

Question: How do you enable CloudTrail to record AWS API activities?
Procedure:

1. Open **AWS CloudTrail**.
2. Select **Trails**.
3. Click **Create trail**.
4. Enter trail name.
5. Select management events.
6. Select Read and/or Write events.
7. Select/create an S3 bucket for logs.
8. Configure additional options if required.
9. Create the trail.
10. Open **Event History** to view activities.
11. Filter events by username, event name, resource or time.
    Remember: CloudTrail = Who did what?

11. CloudWatch

Question: How do you create an alarm when EC2 CPU utilization exceeds 80%?
Procedure:

1. Open **Amazon CloudWatch**.
2. Go to **Metrics**.
3. Select **EC2**.
4. Select the required instance.
5. Select **CPUUtilization**.
6. Click **Create alarm**.
7. Set condition: CPU > 80%.
8. Select evaluation period.
9. Configure an action such as SNS notification.
10. Create the alarm.
11. CloudWatch triggers the action when the threshold is crossed.
    Flow: EC2 → CloudWatch → CPU > 80% → Alarm → SNS

12. CI/CD

Question: How do you create a basic CI/CD pipeline in AWS?
Procedure:

1. Store application code in GitHub or another supported source.
2. Open **AWS CodeBuild**.
3. Create a build project.
4. Connect the source repository.
5. Configure the build environment.
6. Define build commands using buildspec.yml.
7. Open **AWS CodePipeline**.
8. Click **Create pipeline**.
9. Configure the **Source stage**.
10. Configure the **Build stage** using CodeBuild.
11. Add a deployment stage.
12. Select CodeDeploy, ECS or another deployment target.
13. Create the pipeline.
14. Push a new code change.

15. The pipeline automatically retrieves, builds, tests and deploys the application.
    Flow: Developer → GitHub → CodePipeline → CodeBuild → Deployment → Application

1. How do you create a 3-tier VPC architecture step-by-step?

Scenario: A company wants to deploy a web application with separate Web, Application and Database layers.
Procedure:

1. Open **AWS VPC Console → Create VPC**.
2. Create VPC with CIDR 10.0.0.0/16.
3. Create a **Public Web Subnet**, e.g. 10.0.1.0/24.
4. Create a **Private Application Subnet**, e.g. 10.0.2.0/24.
5. Create a **Private Database Subnet**, e.g. 10.0.3.0/24.
6. Create an **Internet Gateway (IGW)** and attach it to the VPC.
7. Create a **Public Route Table**.
8. Add route 0.0.0.0/0 → Internet Gateway.
9. Associate the Public Route Table with the Web subnet.
10. Create a **NAT Gateway** inside the Public subnet.
11. Create a **Private Route Table**.
12. Add route 0.0.0.0/0 → NAT Gateway.
13. Associate the Private Route Table with the Application subnet.
14. Keep the Database subnet private without a direct Internet route.
15. Create appropriate **Security Groups** for Web, Application and Database tiers.
16. Launch EC2 instances in the Web/Application subnets and RDS in the Database subnet.
    Architecture: Internet → IGW → Web Subnet → App Subnet → DB Subnet

2. How do you configure a NAT Gateway for a private EC2 instance?

Scenario: An EC2 instance in a private subnet needs Internet access to download software updates, but should not be directly accessible from the Internet.
Procedure:

1. Create a VPC and public/private subnets.
2. Create and attach an **Internet Gateway**.
3. Create a Public Route Table.
4. Add 0.0.0.0/0 → Internet Gateway.
5. Associate the Public Route Table with the public subnet.

6. Go to **VPC → NAT Gateways → Create NAT Gateway**.
7. Select the **Public Subnet**.
8. Allocate an **Elastic IP**.
9. Create the NAT Gateway.
10. Open the Private Route Table.
11. Add 0.0.0.0/0 → NAT Gateway.
12. Associate the Private Route Table with the private subnet.
13. Launch the EC2 instance in the private subnet.
14. Test Internet access from the private EC2.
    Flow: Private EC2 → NAT Gateway → IGW → Internet
    Important: NAT Gateway is placed in a **public subnet**, while the EC2 using it can remain in a **private subnet**.

3. How do you create and secure an RDS database?

Scenario: A company wants to create a MySQL database for its application.
Procedure:

1. Open **Amazon RDS → Create database**.
2. Select **Standard Create**.
3. Select **MySQL**.
4. Choose the required database version.
5. Enter DB identifier, username and password.
6. Select the required DB instance class and storage.
7. Select the required **VPC**.
8. Select/create a **DB Subnet Group** containing private subnets.
9. Create a Security Group for RDS.
10. Allow inbound MySQL traffic on port **3306** only from the application's Security Group.
11. Enable **KMS encryption** if required.
12. Configure backups and monitoring.
13. Click **Create database**.
14. Wait until the status becomes **Available**.
15. Copy the RDS endpoint.
16. Connect the application to the endpoint.
    Security: Application EC2 → Security Group → RDS
    Important: Avoid allowing 0.0.0.0/0 on port 3306 for a production database.

4. How do you create a DynamoDB table and perform CRUD operations?

Scenario: A student management application needs a NoSQL database.
Procedure:

1. Open **DynamoDB**.
2. Click **Create table**.
3. Enter table name, e.g. Students.
4. Create a partition key such as StudentID.
5. Select the key type, e.g. String.
6. Select **On-demand** or **Provisioned** capacity.
7. Click **Create table**.
8. Open **Explore table items**.
9. Click **Create item**.
10. Add attributes such as Name, Course and Marks.
11. Save the item.
    CRUD operations:

- **Create:** Add a new item.
- **Read:** Get or query an item using its key.
- **Update:** Modify existing attributes.
- **Delete:** Remove an item.
  Example item:
  StudentID = S101, Name = Rahul, Course = CSE, Marks = 85

5. How do you identify who deleted an EC2 instance using CloudTrail?

Scenario: An EC2 instance was accidentally deleted and the administrator wants to find who performed the action.
Procedure:

1. Open **AWS CloudTrail**.
2. Go to **Event history**.
3. Select the required time period.
4. Search/filter for the event name **TerminateInstances**.
5. Open the relevant event.
6. Check the **Username/UserIdentity** information.
7. Check the event time and source IP.
8. Check which EC2 instance was terminated.

9. Review the event details to determine whether the action came from a user, role or AWS service.
   Remember: CloudTrail records API activity and helps answer "Who did what, when and from where?"

6. How do you create a CloudWatch alarm for high CPU utilization?

Scenario: An administrator wants an alert when an EC2 instance's CPU usage exceeds 80%.
Procedure:

1. Open **Amazon CloudWatch**.
2. Select **Metrics**.
3. Select **EC2**.
4. Select the required instance.
5. Select **CPUUtilization**.
6. Click **Create alarm**.
7. Set condition to **CPU > 80%**.
8. Select the required evaluation period.
9. Select an action such as **SNS notification**.
10. Choose/create an SNS topic and email subscription.
11. Create the alarm.
12. When CPU exceeds the threshold, CloudWatch changes the alarm state and triggers the configured action.
    Flow: EC2 → CloudWatch Metric → Alarm → SNS → Email

7. How do you create and deploy an API using API Gateway?

Scenario: A student application needs a GET API that returns student information.
Procedure:

1. Open **API Gateway**.
2. Click **Create API**.
3. Select **REST API** and click **Build**.
4. Enter API name, e.g. StudentAPI.
5. Create a resource /students.
6. Select /students → Create Method → GET.
7. Select **Lambda Function** as the integration.
8. Select the required Lambda function.
9. Save the method and allow API Gateway to invoke Lambda if prompted.

10. Test the GET method.
11. Click **Deploy API**.
12. Create a stage such as dev.
13. Copy the **Invoke URL**.
14. Test the URL using Postman or a browser.
    Flow: Client → API Gateway → Lambda → Database → Response

8. How do you automate infrastructure using CloudFormation?

Scenario: A company wants to create the same VPC, subnet and EC2 configuration repeatedly without manually creating each resource.
Procedure:

1. Create a CloudFormation template in **YAML or JSON**.
2. Define resources such as VPC, Subnet, Route Table, EC2 and Security Group in the template.
3. Open **AWS CloudFormation**.
4. Click **Create stack**.
5. Select **Template is ready**.
6. Upload the template.
7. Enter a stack name.
8. Provide parameters if required.
9. Review the resources that CloudFormation will create.
10. Acknowledge IAM capabilities if required.
11. Click **Create stack**.
12. Monitor the **Events** tab.
13. Check the **Resources** tab after successful creation.
14. CloudFormation creates and manages the resources defined in the template.
    Remember: Template → Stack → AWS Resources
    Main advantage: Infrastructure can be created consistently and repeatedly.

9. How do you encrypt an RDS database using KMS?

Scenario: A hospital wants to encrypt sensitive patient data stored in RDS.
Procedure:

1. Open **AWS KMS**.
2. Select **Customer managed keys**.
3. Click **Create key**.
4. Select a **Symmetric** key.

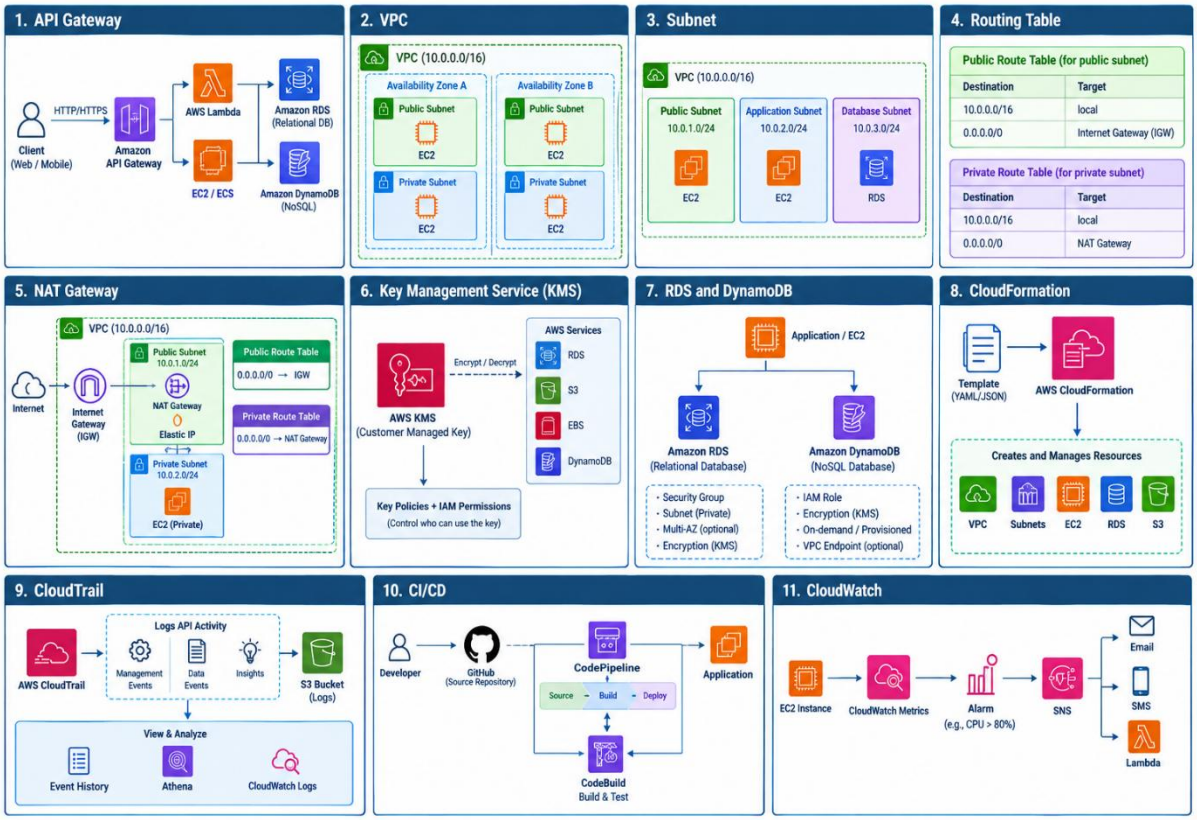
5. Give it an alias such as RDS-Encryption-Key.
6. Configure key administrators and users/roles.
7. Review and create the key.
8. Open **Amazon RDS → Create database**.
9. Select the required database engine.
10. In the encryption section, enable **Storage encryption**.
11. Select the KMS key created earlier.
12. Complete the RDS configuration.
13. Create the database.
14. Ensure the required IAM roles/users have permission to use the KMS key.
    Result: RDS storage, automated backups and snapshots can use encryption associated with the selected KMS key.
    Important: KMS manages the encryption keys; IAM and the KMS key policy control who can use the key.

10. How do you create a CI/CD pipeline that automatically builds and deploys an application?

Scenario: Developers want every code change to automatically build, test and deploy their application.
Procedure:

1. Store the application source code in **GitHub** or another supported repository.
2. Create a **buildspec.yml** file containing build/test commands.
3. Open **AWS CodeBuild**.
4. Create a CodeBuild project.
5. Connect the source repository.
6. Select the required build environment.
7. Configure build commands.
8. Test the CodeBuild project.
9. Open **AWS CodePipeline**.
10. Click **Create pipeline**.
11. Configure the **Source stage**.
12. Configure the **Build stage** using CodeBuild.
13. Add a **Deploy stage**.
14. Select the required deployment service, such as CodeDeploy, ECS or another supported target.

15. Create the pipeline.
16. Push a new code change to the repository.
17. CodePipeline automatically starts the pipeline.
18. The code is downloaded, built/tested and deployed.
    Flow: Developer → GitHub → CodePipeline → CodeBuild → Deploy → Application



1. API Gateway Architecture

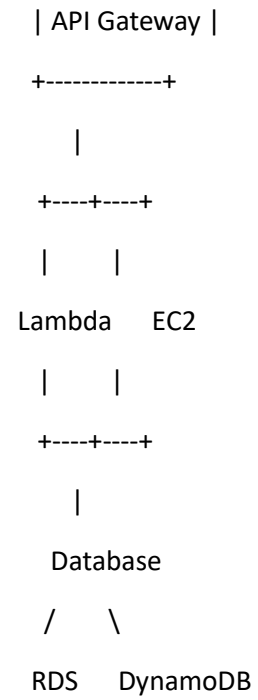
Client

|

HTTP/HTTPS

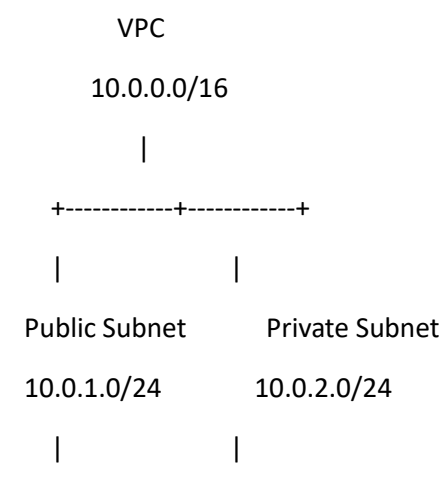
|

+----------+

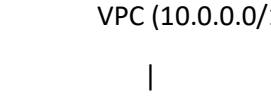


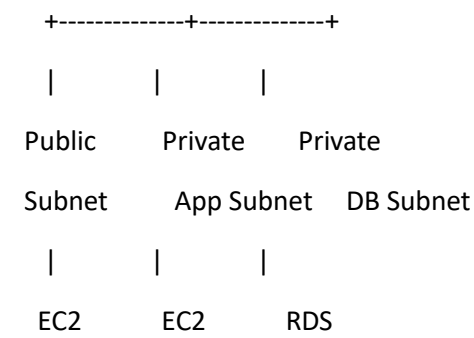
Write below diagram:
Client → API Gateway → Backend → Database → Response

2. VPC Architecture

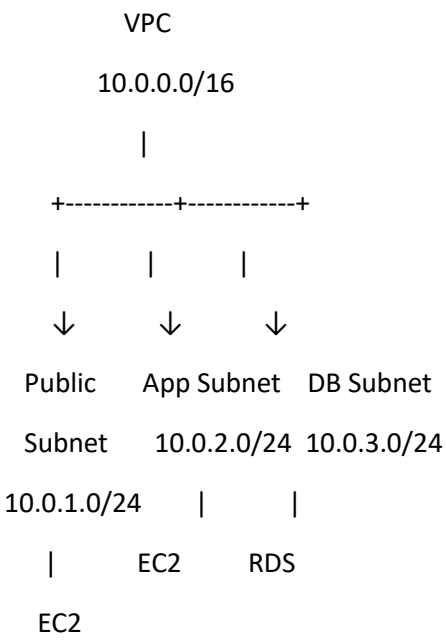


For a better 3-tier diagram:





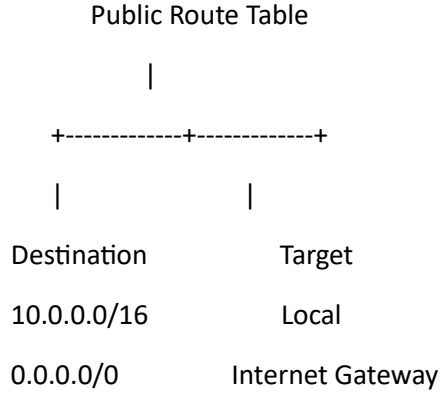
3. Subnet Architecture



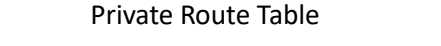
Remember:
VPC → Subnets → AWS Resources

4. Routing Table Architecture

Public Route Table



Private Route Table



|

+-------------+-------------+

| |

Destination Target

10.0.0.0/16 Local

0.0.0.0/0 NAT Gateway

Exam flow:

Public Subnet → Route Table → Internet Gateway → Internet

Private Subnet → Route Table → NAT Gateway → Internet Gateway → Internet

5. NAT Gateway Architecture

Internet

|

Internet Gateway

|

Public Subnet

|

NAT Gateway

|

Private Subnet

|

Private EC2

Important: NAT Gateway is placed in the **public subnet**.
Main purpose: Allows private resources to access the Internet without allowing direct Internet connections to them.

6. AWS KMS Architecture

AWS KMS

Customer Managed Key

|

Encrypt / Decrypt

|

+---------+---------+

| | |

RDS S3 EBS

|

Encrypted Data

More complete:

IAM User/Role

|

KMS Key Policy

|

AWS KMS

|

Encryption / Decryption

|

+----------+----------+

| | |

RDS S3 DynamoDB

Remember:
KMS → Encryption Key → AWS Resource → Encrypted Data

7. RDS Architecture

Application

|

EC2

|

Security Group

|

Private Subnet

|

RDS

MySQL/PostgreSQL

|

Storage

For high availability:

Application

|

EC2

|

RDS Primary

|

Synchronous

Replication

|

RDS Standby

(Another AZ)

Exam flow:
Application → EC2 → Security Group → RDS

8. DynamoDB Architecture

Application

|

API Gateway

|

Lambda

|

DynamoDB

|

+---------+---------+

| | |

Item 1 Item 2 Item 3

Example:

DynamoDB: Students

|

+------+----------------+

| StudentID | Name | Marks |

+-----------+------+-------+

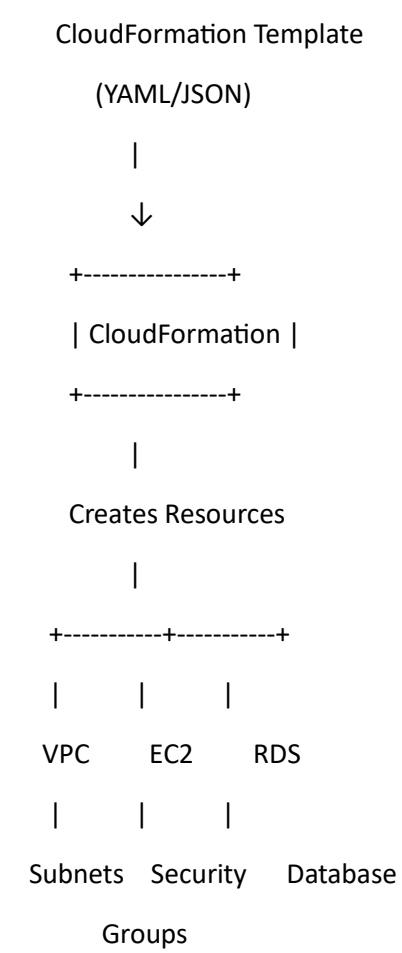
| S101 | Ravi | 85 |

| S102 | Amit | 90 |

+-----------+------+-------+

Remember:
Application → API/Lambda → DynamoDB

9. CloudFormation Architecture

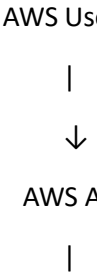


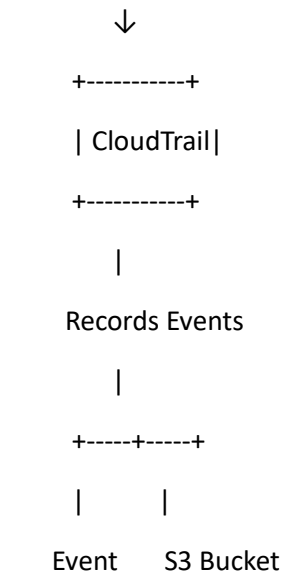
Key concept:

Template → CloudFormation Stack → AWS Resources

This is the easiest diagram to draw in an exam.

10. CloudTrail Architecture





Example:

User → Delete EC2

↓

CloudTrail

↓

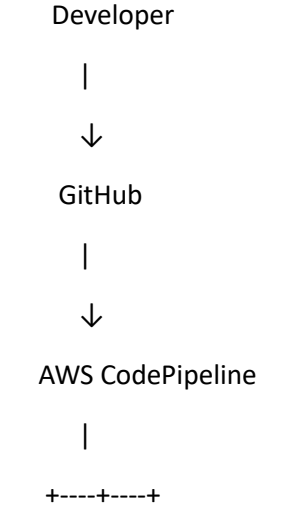
Records API Event

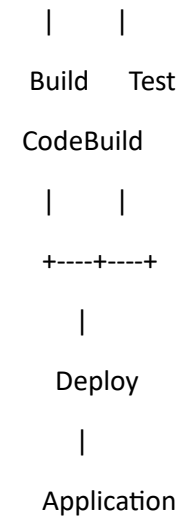
↓

Who + What + When + Where

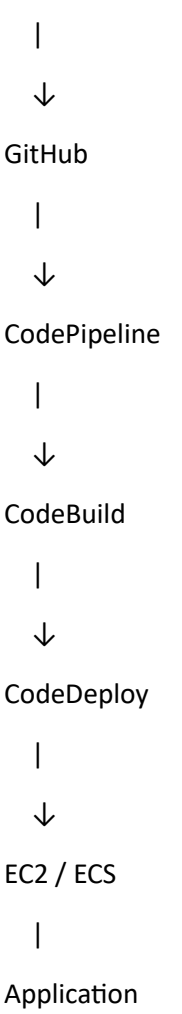
Remember:
CloudTrail = Records AWS activity

11. CI/CD Architecture



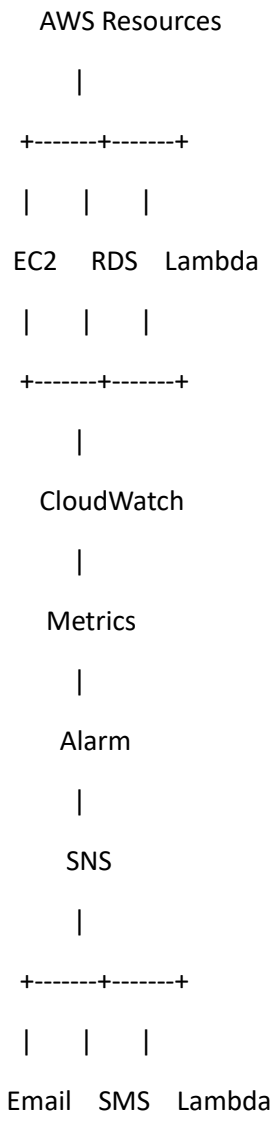


A simpler version:



Remember:
Source → Build → Test → Deploy

12. CloudWatch Architecture



Example:

EC2

|

↓

CPU Utilization

|

↓

CloudWatch

|

↓

CPU > 80%

|

↓

Alarm

|

↓

SNS Notification

1. Secure and Highly Available 3-Tier AWS Architecture

Question

An online shopping company wants a secure, highly available 3-tier application on AWS with Web, Application, and Database tiers. The VPC has **256 IPv4 addresses (/24)** and must be divided into **equal-sized subnetworks** across two Availability Zones.

Answer

1. Proposed Architecture

The application will use:

- **Web Tier:** Public subnets
- **Application Tier:** Private subnets
- **Database Tier:** Private database subnets
- **Two Availability Zones:** AZ-1 and AZ-2
- **Internet Gateway:** Internet access for public resources
- **NAT Gateways:** Outbound Internet access for private resources
- **Route Tables:** Control traffic flow
- **Security Groups:** Control communication between tiers

2. VPC and Subnet Calculation

Given:

VPC = 192.168.10.0/24

Total addresses = 256

There are **3 tiers × 2 Availability Zones = 6 subnets** required.

Since all subnetworks must be equal-sized, divide the /24 into **8 equal /27 subnets**.

/24 → /27

Each /27 provides:

32 total IP addresses

30 usable host IP addresses

We need only 6 of the 8 available /27 subnets.

3. Subnet Allocation

Tier	AZ-1	AZ-2
Web	192.168.10.0/27	192.168.10.32/27
Application	192.168.10.64/27	192.168.10.96/27
Database	192.168.10.128/27	192.168.10.160/27
Spare	192.168.10.192/27	192.168.10.224/27

The last two subnets can be reserved for future requirements.

Important: In AWS, a subnet has 5 IP addresses reserved by AWS, so a /27 subnet has **27 usable IPv4 addresses for resources**, rather than 30.

---

4. Web Tier

The Web Tier is placed in public subnets:

AZ-1 → 192.168.10.0/27

AZ-2 → 192.168.10.32/27

Web servers need Internet accessibility.

The public route table contains:

Destination Target

192.168.10.0/24 local

0.0.0.0/0 Internet Gateway

Traffic:

Internet

↓

Internet Gateway

↓

Web Servers

An Application Load Balancer can be placed in the public subnets to distribute traffic between web servers.

---

5. Application Tier

Application servers are placed in private subnets:

AZ-1 → 192.168.10.64/27

AZ-2 → 192.168.10.96/27

They do not receive direct Internet traffic.

Their route table can contain:

Destination Target

192.168.10.0/24 local

0.0.0.0/0 NAT Gateway

Therefore:

Application Server

↓

NAT Gateway

↓

Internet Gateway

↓

Internet

This allows software updates and outbound connections without making the application servers publicly accessible.

---

6. Database Tier

Database servers/RDS are placed in private database subnets:

AZ-1 → 192.168.10.128/27

AZ-2 → 192.168.10.160/27

The database should not have a route directly to an Internet Gateway.

For maximum isolation, the database route table should contain only the local VPC route:

Destination Target

192.168.10.0/24 local

The database should be configured as **not publicly accessible**.

---

7. NAT Gateway

For high availability, use a NAT Gateway in each Availability Zone.

AZ-1:

Private Application → NAT Gateway 1 → Internet Gateway

AZ-2:

Private Application → NAT Gateway 2 → Internet Gateway

This avoids depending on a single NAT Gateway in one AZ.

If database servers genuinely need outbound Internet access for a requirement, their private route can use the appropriate NAT Gateway. Otherwise, keeping the database tier without a default Internet route provides stronger isolation.

---

8. Security Groups

Web Security Group

Allow:

HTTP (80) → Internet

HTTPS (443) → Internet

Allow outbound traffic as required.

Application Security Group

Allow application traffic **only from the Web/ALB security group**.

Example:

Web/ALB SG → Application SG

Port 8080

Do not allow application servers to receive traffic directly from the Internet.

Database Security Group

Allow database traffic only from the Application Security Group.

For example:

Application SG → Database SG

Port 3306 (MySQL)

or:

Application SG → Database SG

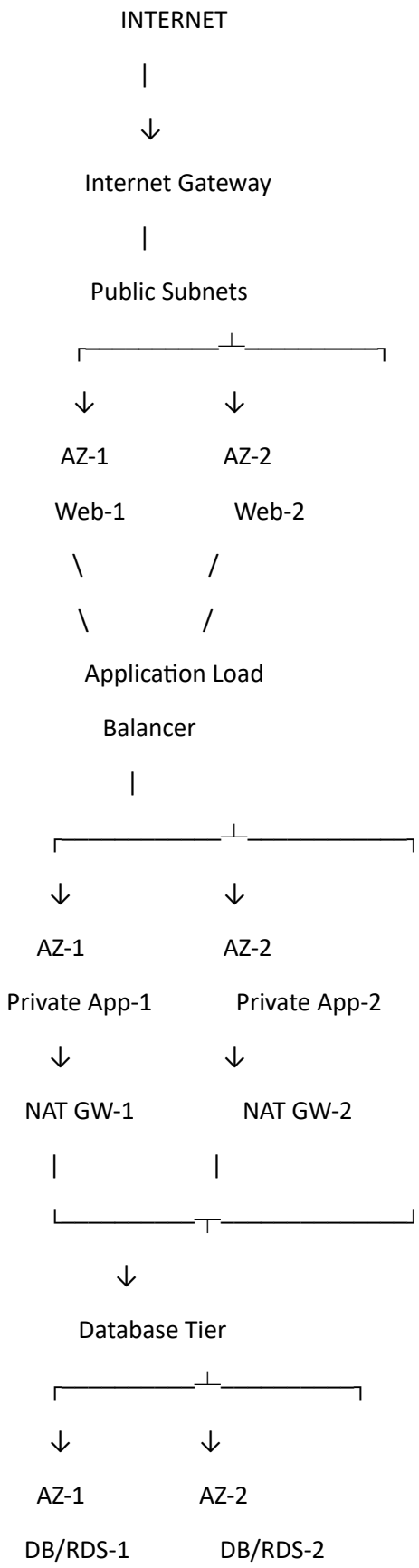
Port 5432 (PostgreSQL)

Do not allow:

Internet → Database

---

9. Final Architecture



Conclusion

The proposed architecture provides **security, high availability, and network isolation**. Public web servers are accessible through the Internet Gateway, while application and database servers remain private. NAT Gateways provide controlled outbound Internet access, and security groups restrict communication between tiers. Deploying each tier across two Availability Zones improves availability.

---

2. REST API for E-Commerce Website Using API Gateway

Question

Design and implement a REST API using AWS API Gateway for an e-commerce website. The website should dynamically retrieve product information. The company also wants to introduce newer API versions without affecting the existing production version.

Answer

1. Architecture

A suitable architecture is:

Customer / Website

|

↓

API Gateway

|

↓

Lambda

|

↓

DynamoDB / RDS

API Gateway acts as the entry point for the website.

---

2. Create REST API

Create a REST API in **Amazon API Gateway**.

Example API:

E-Commerce Product API

Create the following resources:

/products

/products/{productId}

---

3. API Methods

GET /products

Returns all or available products.

Example:

GET /products

Response:


```
{
  "products": [
    {
      "id": "P101",
      "name": "Laptop",
      "price": 55000
    },
    {
      "id": "P102",
      "name": "Mobile",
      "price": 25000
    }
  ]
}
```


GET /products/{productId}

Returns information about a particular product.

GET /products/P101

---

4. Backend Integration

API Gateway can invoke an AWS Lambda function.

GET /products

↓

API Gateway

↓

Lambda

↓

Database

↓

Product Information

↓

API Gateway

↓

Website

Lambda retrieves the latest product information from the database and returns it to API Gateway.

---

5. API Versioning

The company wants to introduce a new API version without immediately affecting existing users.

Use separate API versions/stages such as:

/v1/products

/v2/products

Example:

Production Website → API v1

New Website → API v2

Existing customers can continue using v1 while v2 is tested.

---

6. Stages

API Gateway stages can be used to separate environments such as:

Development

Testing

Production

For example:

API

|

├— dev

├— test

└— prod

This prevents development changes from immediately affecting production.

---

7. Security

API Gateway can be secured using:

- IAM authorization
- Lambda authorizers
- Amazon Cognito
- API keys/usage plans where appropriate
- HTTPS

Only authorized clients should be allowed to access protected APIs.

---

8. Throttling

The company can configure throttling to prevent excessive requests from overwhelming the backend.

Example:

Maximum requests → configured limit

Requests exceeding configured limits can be throttled.

---

9. Monitoring

API Gateway can integrate with **CloudWatch**.

Important metrics include:

- Number of requests
- Latency
- 4xx errors
- 5xx errors

This helps identify API performance and failures.

---

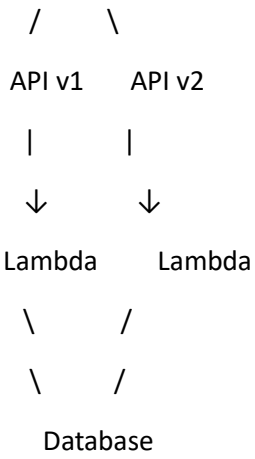
10. Final Architecture

E-Commerce Website

|

↓

API Gateway



Conclusion

API Gateway provides a managed and scalable way to expose product APIs. Using separate API versions/stages allows new functionality to be developed and tested without immediately disrupting the existing production API.

---

3. Role-Based Access Control Using AWS IAM

Question

An online shopping company has Developers, DBAs, Security Auditors, and Cloud Administrators. Design an IAM solution satisfying:

- Developers → EC2 development resources
- DBAs → RDS resources
- Auditors → Read-only access
- Administrators → Full access
- EC2 → S3 through IAM Role
- Lambda → CloudWatch Logs through IAM Role

Answer

1. Introduction

AWS Identity and Access Management (IAM) controls **who can access AWS resources and what actions they can perform**.

The company should follow:

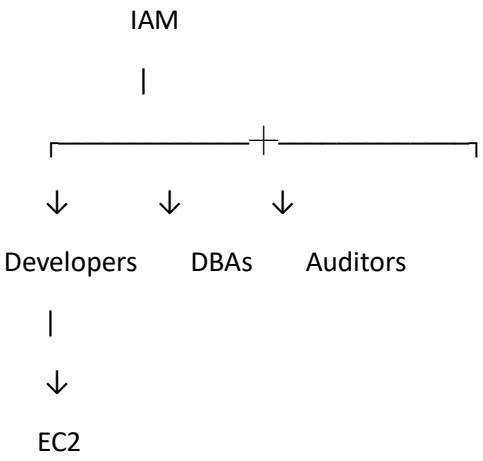
Principle of Least Privilege

Users should receive only the permissions required for their jobs.

---

2. IAM Groups/Roles

A suitable structure is:



Administrators can have a separate administrative role/group.

---

3. Developers

Developers require access to EC2 development resources.

Create:

Developers Group

Attach a policy allowing required EC2 actions.

For example, permissions can include actions needed to:

- View EC2 resources
- Start instances
- Stop instances
- Reboot development instances

Permissions should be restricted to development resources where possible.



---

4. Database Administrators

Create:

DBA Group/Role

Give required RDS permissions.

DBAs may need to:

- View RDS instances
- Create/modify database instances where required
- Create snapshots
- Monitor database configuration

Access should be restricted to the organization's RDS resources as appropriate.

DBA

↓

DBA IAM Role

↓

RDS

---

5. Security Auditors

Auditors need to inspect resources without modifying them.

Create:

Auditors Group/Role

Attach appropriate **read-only** permissions.

They should be able to:

- View configurations
- Inspect resources
- Review security information
- Access relevant logs according to policy

But should not be able to:

Create

Modify

Delete

resources.

---

6. Cloud Administrators

Administrators require broad management permissions.

Create:

Administrators Role/Group

They can be granted appropriate administrative permissions, subject to the organization's security policy.

Administrator

↓

Admin Role

↓

AWS Resources

Because administrative permissions are powerful, they should be used carefully and protected with strong authentication such as MFA.

---

7. EC2 → S3 IAM Role

EC2 instances should **not** store long-term AWS access keys.

Instead, create an IAM role for EC2.

EC2 Instance

↓

IAM Role

↓

S3

Attach a policy granting only the required S3 operations.

For example, if the application only needs to read objects, grant appropriate read permissions rather than full S3 access.

The EC2 instance receives temporary credentials through the role.

Benefit

- No hard-coded access keys
- Temporary credentials
- Easier credential management
- Least-privilege access

---

8. Lambda → CloudWatch Logs IAM Role

Lambda functions need permissions to write logs to CloudWatch Logs.

Create an execution role:

Lambda

↓

Lambda Execution Role

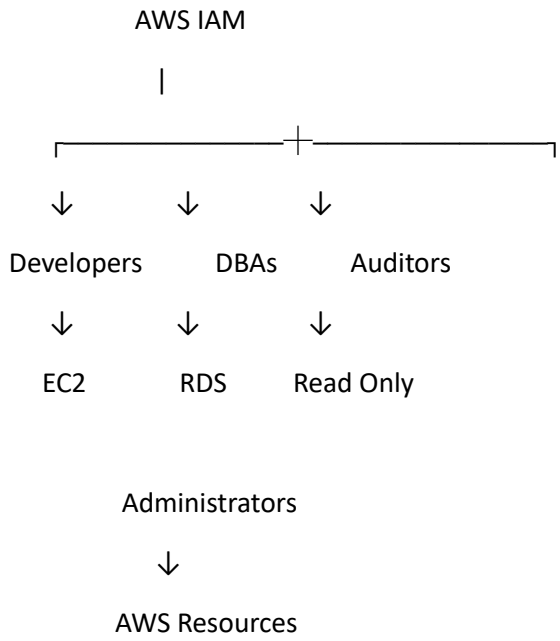
↓

CloudWatch Logs

The role should contain the required CloudWatch Logs permissions, such as permissions to create log groups/streams and put log events, typically through an appropriate AWS-managed or customer-managed policy.

---

9. IAM Structure



EC2 ———→ IAM Role ———→ S3

Lambda —→ Execution Role —→ CloudWatch Logs

---

10. Security Best Practices

The company should:

1. Follow least privilege.
2. Use IAM roles for AWS services.

3. Avoid hard-coded access keys.
4. Enable MFA for privileged users.
5. Regularly review permissions.
6. Remove unused users and permissions.
7. Use groups/roles instead of assigning excessive permissions individually.
8. Monitor IAM activity using CloudTrail.

Conclusion

IAM can implement role-based access by giving each team only the permissions required for its responsibilities. Developers receive EC2 access, DBAs receive RDS access, auditors receive read-only permissions, and administrators receive broad administrative access. IAM roles securely provide EC2 access to S3 and Lambda access to CloudWatch Logs without embedding permanent credentials in applications.